DP Take Or Leave with 2D only :

```
dp[0][0][0] = 1 ;
   for (int i = 1; i <= n; i++)
   {
       int cur = i&1;
       int prev = !cur;

       for (int w = 0; w <= trg; w++)
           dp[w][0][cur] = dp[w][1][cur] = 0;

       for (int w = 0; w <= trg; w++)
       {
           // leave
           dp[w][0][cur] += dp[w][0][prev];
           dp[w][1][cur] += dp[w][1][prev];

           // take
           if(w - arr[i-1]>= 0){
               if(arr[i-1]%2 == 0){
                   dp[w][1][cur] += dp[w-arr[i-1]][0][prev];
               }else{
                   dp[w][0][cur] += dp[w-arr[i-1]][0][prev];
                   dp[w][1][cur] += dp[w-arr[i-1]][1][prev];
               }
           }
       }
   }
```

DP Digit :

```
const int N = 20 ;
int dp[N][10][2][2];
int rec (int pos, int last , int lower , int st){
    if(pos == N)return 1 ;
    int &ret = dp[pos][last][lower][st];
    if(~ret) return ret ;
    ret = 0 ;
    int up = lower ? 9 : a[pos]-'0';
    for (int i = 0; i <= up; i++)
    {
        if(last == i && st)continue;
        ret += rec(pos+1 , i , lower || (i < up) , st || (i > 0));
    }
    return ret ;
}
```

iteration over all subsets of the mask :

```
// iterate over all the masks
for (int mask = 0; mask < (1<<n); mask++){
    F[mask] = A[0];
    // iterate over all the subsets of the mask
    for(int i = mask; i > 0; i = (i-1) & mask){
        F[mask] += A[i];
    }
}
```

SOS DP :

```
    // submask
    for (int i = 0; i < N; i++) dp[i] = freq[i];

    for (int bit = 0; bit < K; bit++) {
        for (int mask = 0; mask < N; mask++) {
            if (mask >> bit & 1) {
                dp[mask] += dp[mask ^ (1 << bit)];
            }
        }
    }

    // supermask
    for (int i = 0; i < N; i++) dp2[i] = freq[i];

    for (int bit = 0; bit < K; bit++) {
```

```
        for (int mask = 0; mask < N; mask++) {
            if (!(mask >> bit & 1 )) {
                dp2[mask] += dp2[mask ^ (1 << bit)];
            }
        }
    }


    // share at lest one bit
    int full = (1 << K) - 1;

    for (int i = 0; i < N; i++) {
        int comp = full ^ i;
        dp3[i] = n - dp[comp];
    }
```